

Lab 14: Web Frontend Development – AI-Assisted HTML/CSS/JS Generation

Task 1: Responsive Homepage Layout for a Startup Website

Prompt

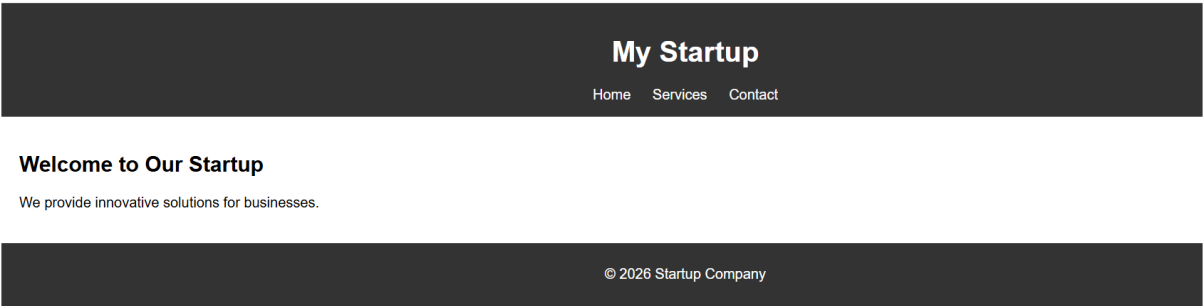
1. Create a simple startup company homepage using HTML with header, main content, and footer sections clearly separated.
2. Add navigation links inside the header so users can move to different sections of the website easily.
3. Write responsive CSS using media queries so the layout adjusts automatically for desktop, tablet, and mobile devices.
4. Make sure the design becomes a single column on small screens and a wider layout on large screens.
5. Keep the code clean, simple, and easy to understand for beginners.

Code:

```

1  <html>
2  <html>
3  <head>
4  <style>
5  header{
6  padding:10px;
7  text-align:center;
8  }
9
10 nav a{
11 color:blue;
12 margin:5px;
13 text-decoration:none;
14 }
15
16 main{
17 padding:20px;
18 }
19
20 footer{
21 background-color:#f3f3f3;
22 color:blue;
23 text-align:center;
24 padding:10px;
25 }
26
27 /* Tablet */
28 @media (max-width:768px){
29 header{
30 font-size:18px;
31 }
32 }
33
34 /* Mobile */
35 @media (max-width:480px){
36 nav a{
37 display:block;
38 margin:5px;
39 }
40 }
41
42 </style>
43 </head>
44 <body>
45
46 <header>
47 <h1>My Startup</h1>
48 <nav>
49 <a href="#">Home</a>
50 <a href="#">Services</a>
51 <a href="#">Contact</a>
52 </nav>
53 </header>
54
55 <main>
56 <h2>Welcome to Our Startup</h2>
57 <p>We provide innovative solutions for businesses.</p>
58 </main>
59
60 <footer>
61 <p>© 2024 Startup Company</p>
62 </footer>
63
64 </body>
65 </html>
```

Output:



Explanation:

- 1. HTML is used to create the page structure with header, main content, and footer.
- 2. CSS is used to style the layout and make the design look clean.
- 3. Media queries help the page adjust for different screen sizes like mobile and tablet.
- 4. This makes the website responsive and easy to view on any device.

Task 2: Interactive Call-to-Action Button

Prompt

- 1. Create a web page with a call-to-action button using HTML.
- 2. When the user clicks the button, JavaScript should display a message or alert.
- 3. The button should respond immediately without refreshing the page.
- 4. Add meaningful comments in JavaScript to explain the logic clearly.
- 5. Keep the code simple so beginners can easily understand how the interaction works.

Code:

```
<!DOCTYPE html>
<html>
  <head>
    <title>CTA Button</title>
  </head>

  <body>

    <h2>Join Our Service</h2>

    <button onclick="showMessage()">Click Here</button>

    <script>

      // Function runs when button is clicked
      function showMessage(){

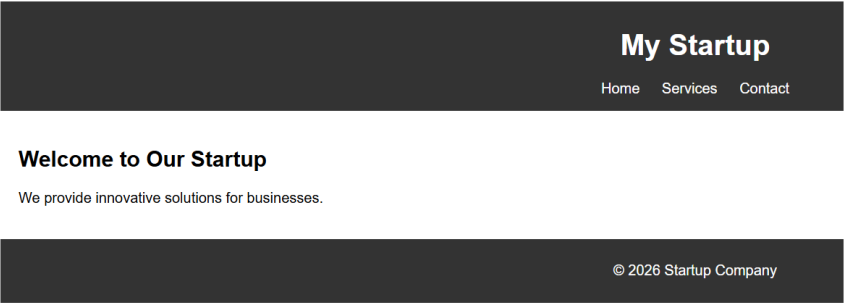
        // Display alert message to user
        alert("Thank you for clicking the button!");

      }

    </script>

  </body>
</html>
```

Output:



Join Our Service

[Click Here](#)

Explanation:

1. HTML creates the button on the webpage.
2. JavaScript function runs when the button is clicked.
3. The alert message shows a response to the user.
4. This interaction improves user engagement on websites.

Task 3: Contact Form with Client-Side Validation

Prompt

1. Create a contact form using HTML with fields for name, email, and message.
2. Use JavaScript to validate the form before submitting the data.
3. The form should not allow empty fields to be submitted.
4. The program should check if the email format is correct.
5. Display clear error messages for invalid inputs and success message for valid submission.

Code:

```
111 </head>
112
113 <body>
114
115 <h2>Contact Us</h2>
116
117 <form onsubmit="return validateForm()">
118
119   Name:<br>
120   <input type="text" id="name"><br><br>
121
122   Email:<br>
123   <input type="text" id="email"><br><br>
124
125   Message:<br>
126   <textarea id="message"></textarea><br><br>
127
128   <input type="submit" value="Submit">
129
130   <p id="error" style="color: red;"></p>
131
132 </form>
133
134 <script>
135
136 function validateForm(){
137
138   var name=document.getElementById("name").value;
139   var email=document.getElementById("email").value;
140   var message=document.getElementById("message").value;
141
142   if(name=="" || email=="" || message==""){
143     document.getElementById("error").innerHTML="All fields are required";
144     return false;
145   }
146
147   if(!email.includes("@")){
148     document.getElementById("error").innerHTML="Enter valid email";
149     return false;
150   }
151
152   alert("Form submitted successfully!");
153   return true;
154 }
```

Output:

Welcome to Our Startup

We provide innovative solutions for businesses.



-->

Contact Us

Name:

Email:

Message:

Submit

Explanation:

1. HTML creates the contact form structure.
2. JavaScript checks if the fields are empty before submitting.
3. It also checks if the email contains a valid format.
4. Error messages guide the user to correct the input.

Task 4: Dynamic List Management for Product Page

Prompt

1. Create a simple HTML list that displays product items.
2. Add two buttons named Add Item and Remove Item.
3. Use JavaScript to dynamically add new items to the list.
4. Use another JavaScript function to remove the last item from the list.
5. Ensure the changes happen instantly without refreshing the webpage.

Code:

```
162  -->
163  <!DOCTYPE html>
164  <html>
165  <head>
166  <title>Product List</title>
167  </head>
168
169  <body>
170
171  <h2>Product List</h2>
172
173  <ul id="list">
174  <li>Product 1</li>
175  <li>Product 2</li>
176  </ul>
177
178  <button onclick="addItem()">Add Item</button>
179  <button onclick="removeItem()">Remove Item</button>
180
181  <script>
182
183  function addItem(){
184
185  var ul=document.getElementById("list");
186
187  var li=document.createElement("li");
188
189  li.textContent="New Product";
190
191  ul.appendChild(li);
192
193  }
194
195  function removeItem(){
196
197  var ul=document.getElementById("list");
198
199  ul.removeChild(ul.lastElementChild);
200
201  }
202
203  </script>
204
205  </body>
206  </html>
```

Output:

Welcome to Our Startup

We provide innovative solutions for businesses.

© 2026 Startup Company

Product List

- Product 1
- Product 2

Add Item Remove Item

Explanation:

1. HTML creates the list and buttons.
2. JavaScript adds new list items using DOM manipulation.
3. Another function removes the last item from the list.
4. The webpage updates dynamically without reload.

Task 5: Modal Popup for User Notifications

Prompt

1. Create a modal popup window using HTML structure.

2. Add CSS styling with a semi-transparent background overlay.
3. Use JavaScript to open the modal when clicking the Open Modal button.
4. Add options to close the modal using a close button or clicking outside the popup.
5. Ensure the popup appears clearly above the page content.

Code:

```
<> web1.html > html
210 <html>
211 <head>
214 <style>
216 .modal{
218 position:fixed;
219 top:0;
220 left:0;
221 width:100%;
222 height:100%;
223 background: rgba(0,0,0,0.5);
224 }
225
226 .modal-content{
227 background: white;
228 padding:20px;
229 margin:15% auto;
230 width:300px;
231 text-align:center;
232 }
233
234 </style>
235
236 </head>
237
238 <body>
239
240 <button onclick="openModal()">Open Modal</button>
241
242 <div id="myModal" class="modal">
243
244 <div class="modal-content">
245
246 <p>This is a notification popup</p>
247
248 <button onclick="closeModal()">Close</button>
249
250 </div>
251
252 </div>
253
254 <script>
255
256 function openModal(){
257 document.getElementById("myModal").style.display="block";
258 }
259
260 function closeModal(){
261 document.getElementById("myModal").style.display="none";
262 }
263
264 </script>
265
266 </body>
267 </html>
```

Output:



Explanation:

1. HTML creates the modal popup structure.
2. CSS styles the popup and adds a transparent overlay background.
3. JavaScript controls opening and closing the modal window.
4. This feature is commonly used for notifications and alerts.